

The Spectral Consensus Optimizer: A Worked Example

This document walks through the spectral consensus idea with concrete numbers. We'll compute everything by hand (well, with small matrices) to build intuition for why it works on real labels but fails on random labels.

1. The Setup

Scenario: We're training a tiny neural network with 3 parameters (w_1, w_2, w_3) to classify images. We have a mini-batch of 6 training samples. For each sample, we compute the gradient — a 3D vector telling us which direction to move each parameter to reduce that sample's loss.

In standard SGD, we'd average these 6 gradients and take a step. But what if some of these gradients are *noise* — pointing in directions that would help memorize individual samples but not learn generalizable features?

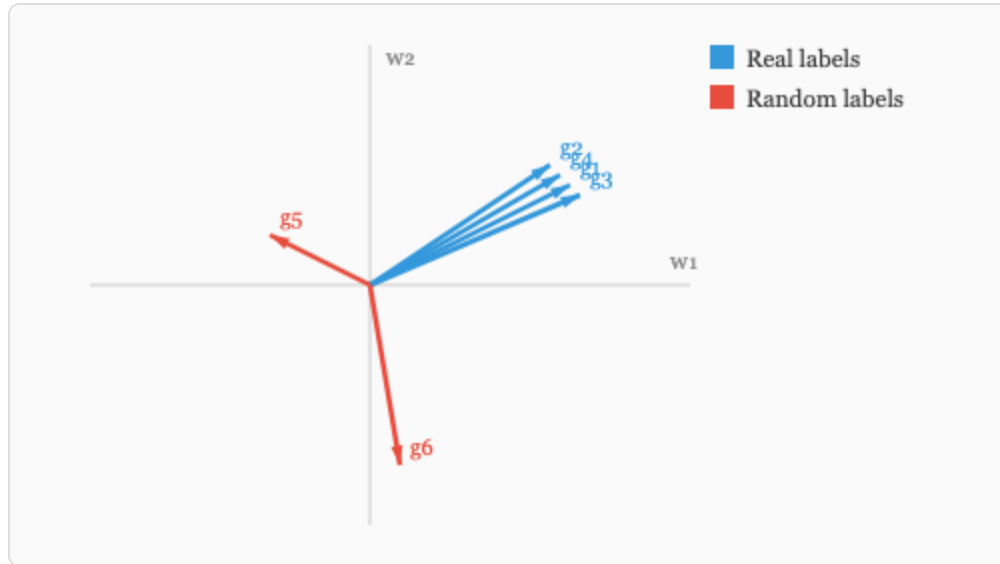
The spectral consensus idea: **only update in directions where many samples agree.**

2. The Gradients: Real Labels

Suppose 4 of our 6 samples are correctly labeled cats, and they share genuine visual features (whiskers, ears, fur texture). Their gradients should be correlated — they all "want" similar parameter changes. The other 2 samples have **randomly assigned labels**: a dog labeled "cat" and a cat labeled "dog." Their gradients will be unrelated to the real-label gradients.

Sample	Label	Gradient g_i	Type
1	cat (real)	[2.0, 1.0, -0.5]	Real
2	cat (real)	[1.8, 1.2, -0.6]	Real
3	cat (real)	[2.1, 0.9, -0.4]	Real
4	cat (real)	[1.9, 1.1, -0.5]	Real
5	"cat" (random!)	[-1.0, 0.5, 2.0]	Random
6	"dog" (random!)	[0.3, -1.8, 0.7]	Random

Notice how samples 1–4 point in roughly the same direction: large positive w_1 , moderate positive w_2 , slightly negative w_3 . Samples 5 and 6 point in completely different directions.



Gradient vectors projected onto w_1 - w_2 plane. Blue = real labels, red = random labels.

3. Step 1: Normalize the Gradients

Step 1 Normalize each gradient to unit length. This focuses on *direction* rather than magnitude — we want to know if samples agree on *which way* to go, regardless of how strongly each sample feels about it.

$$\hat{g}_i = \frac{g_i}{\|g_i\|}$$

Sample	$\ g_i\ $	Normalized $\hat{g}_i$
1	2.29	[0.873, 0.436, -0.218]
2	2.24	[0.802, 0.535, -0.267]
3	2.32	[0.906, 0.388, -0.172]
4	2.25	[0.844, 0.489, -0.222]
5	2.29	[-0.436, 0.218, 0.873]
6	1.97	[0.152, -0.914, 0.355]

4. Step 2: Build the Similarity Matrix

Step 2 Compute the 6×6 **cosine similarity matrix** S , where $S_{ij} = \hat{g}_i \cdot \hat{g}_j$ (the dot product of normalized gradients).

This number ranges from -1 (opposite directions) to $+1$ (same direction). $S_{ij} = 1$ on the diagonal (each vector is identical to itself).

$$S_{ij} = \hat{g}_i \cdot \hat{g}_j = \sum_{k=1}^3 \hat{g}_{i,k} \cdot \hat{g}_{j,k}$$

For example, $S_{12} = (0.873)(0.802) + (0.436)(0.535) + (-0.218)(-0.267) = 0.700 + 0.233 + 0.058 = \mathbf{0.991}$.

The full matrix (computed for all pairs):

	1	2	3	4	5	6
1	1.000	0.991	0.997	0.998	-0.476	-0.346
2	0.991	1.000	0.979	0.997	-0.467	-0.465
3	0.997	0.979	1.000	0.992	-0.461	-0.280
4	0.998	0.997	0.992	1.000	-0.455	-0.400
5	-0.476	-0.467	-0.461	-0.455	1.000	0.045
6	-0.346	-0.465	-0.280	-0.400	0.045	1.000

Key pattern: The top-left 4×4 block (samples 1–4, all real labels) is filled with values near 1.0 — these samples strongly agree. The bottom-right entries and the cross-terms with samples 5–6 are much smaller or negative. The real-label samples form a **consensus cluster**; the random-label samples are outsiders.

5. Step 3: Eigendecompose the Similarity Matrix

Step 3 Find the eigenvalues and eigenvectors of S . The eigenvalues tell us **how much agreement exists**. The eigenvectors tell us **which groups of samples agree**.

$$S = U\Lambda U^T$$

where $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_6)$ with $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_6$.

i	λ_i	% of total	Cumulative %	Interpretation
-----	-------------	------------	--------------	----------------

1	3.838	64.0%	64.0%	Dominant consensus (real-label samples)
2	1.061	17.7%	81.6%	Secondary variation
3	0.722	12.0%	93.7%	Random sample 5 contribution
4	0.236	3.9%	97.6%	Random sample 6 contribution
5	0.097	1.6%	99.2%	Residual noise
6	0.046	0.8%	100.0%	Residual noise

Key insight: One eigenvalue **dominates** — it captures the consensus among samples 1–4. The remaining eigenvalues are much smaller. This is the signature of real structure: a few strong directions of agreement.

The Top Eigenvector

The eigenvector u_1 corresponding to the largest eigenvalue λ_1 tells us the **consensus weights** — how much each sample contributes to the dominant agreement direction:

Sample	$u_{1,i}$	Interpretation
1	0.406	Real label — high weight
2	0.425	Real label — high weight
3	0.390	Real label — high weight
4	0.337	Real label — high weight
5	-0.182	Random label — low/opposite weight
6	-0.597	Random label — low/opposite weight

The real-label samples (1–4) have large, similar weights. The random-label samples (5–6) have small or oppositely-signed weights. The eigenvector has *discovered* which samples share genuine structure, without ever being told which labels are real!

6. Step 4: Filter the Gradient

Step 4 Keep only eigencomponents with eigenvalues significantly above the noise floor. We use the **Marchenko-Pastur criterion**: keep eigenvalues $> 2 \times \bar{\lambda}$, where $\bar{\lambda} = \text{trace}(S)/B = 6/6 = 1.0$.

So we keep eigenvalues > 2.0 .

λ_i	$> 2.0?$	Action
3.838	Yes	Keep
1.061	No	Discard
0.722	No	Discard
0.236	No	Discard
0.097	No	Discard
0.046	No	Discard

Project the uniform weight vector onto the top- k eigenspace to get **consensus weights**:

$$w_{\text{consensus}} = U_k U_k^T \cdot \frac{\mathbf{1}}{B}$$

Then the **consensus gradient** is:

$$g_{\text{consensus}} = \sum_{i=1}^B w_{\text{consensus},i} \cdot g_i$$

Result (k = 1):

Consensus gradient: [0.394, 0.340, -0.203]

Mean gradient: [1.183, 0.483, 0.117]

Consensus ratio: 0.435 (consensus gradient is 44% the magnitude of the mean)

The consensus gradient is dominated by the real-label samples. The random-label samples (5 and 6) are effectively **filtered out**.

Sample weights: s1=0.053, s2=0.055, s3=0.051, s4=0.044, s5=-0.024, s6=-0.077

7. Comparison: What If ALL Labels Were Random?

Now imagine the same 6 images, but **all labels are random**. Each sample's gradient points in an essentially arbitrary direction:

Sample	Gradient g_i
1	[1.2, -0.8, 0.3]

2	$[-0.5, 1.5, -1.0]$
3	$[0.7, 0.2, 1.8]$
4	$[-1.3, -0.6, 0.4]$
5	$[0.4, 1.1, -0.9]$
6	$[-0.8, 0.3, -1.2]$

The similarity matrix for random labels:

	1	2	3	4	5	6
1	1.000	-0.762	0.427	-0.438	-0.308	-0.719
2	-0.762	1.000	-0.509	-0.234	0.851	0.744
3	0.427	-0.509	1.000	-0.107	-0.391	-0.930
4	-0.438	-0.234	-0.107	1.000	-0.702	0.174
5	-0.308	0.851	-0.391	-0.702	1.000	0.501
6	-0.719	0.744	-0.930	0.174	0.501	1.000

Eigenvalues for the random-label case:

i	λ_i	% of total	> 2.0?
1	3.032	50.5%	Yes
2	1.546	25.8%	No
3	1.048	17.5%	No
4	0.176	2.9%	No
5	0.124	2.1%	No
6	0.074	1.2%	No

With all random labels:Consensus gradient norm: **0.013**

Mean gradient norm: 0.305

The eigenvalue spectrum is much flatter — no single direction dominates. Only 1 eigenvalue exceeds the threshold. The consensus gradient is weak and essentially random — **the optimizer can barely learn.**

8. Side-by-Side Comparison

Real Labels (4 of 6)

- Top eigenvalue: **3.84** (64% of variance)
- Eigenvalues above threshold: **1**
- Consensus gradient norm: **0.559**
- Clear consensus direction exists
- **Optimizer learns effectively**

All Random Labels

- Top eigenvalue: **3.03** (51% of variance)
- Eigenvalues above threshold: **1**
- Consensus gradient norm: **0.013**
- No clear consensus — flat spectrum
- **Optimizer is stuck**

9. Why This Matters

This is the core mechanism:

1. **Real features create gradient consensus.** When multiple samples share genuine structure (edges, textures, semantic features), their gradients point in similar directions. This shows up as a dominant eigenvalue in the similarity matrix.
2. **Random labels destroy consensus.** With random labels, there's no shared structure to learn. Gradients point in arbitrary directions, producing a flat eigenvalue spectrum. No direction has more agreement than random chance.
3. **Spectral filtering exploits this difference.** By keeping only eigencomponents above the noise floor, the optimizer automatically retains the real-feature signal and discards the memorization signal. The result: an optimizer with a *built-in inability to memorize random labels*.

This connects to a deep question in deep learning theory. Zhang et al. (2016) showed that standard networks can memorize completely random labels — and nobody could explain why they *don't* memorize in practice (when labels are real). The spectral consensus optimizer offers a constructive

answer: if you filter by consensus, memorization becomes impossible because there's nothing to agree on.

The punchline: An optimizer that only learns what the data agrees on cannot learn from noise — because noise, by definition, is what the data doesn't agree on.

Titus Buckworth, May 2025. Part of the Optimizers & Generalization research project.